

UI Styling

UI styling shapes the storefront's look and feel, including layout, spacing, colors, typography, and interactive states. It provides a consistent, accessible, and responsive shopping experience for your storefront across devices.

Technology Stack

Storefront Next uses [Tailwind CSS](#) (v4) for utility-first styling and [shadcn/ui](#) patterns, and [Radix UI](#) for headless accessible components.

Here are the Tailwind-related dependencies in this package.

Dependency	Version	Purpose
tailwindcss	4.x.x+	Utility-first CSS framework
@radix-ui/*	Various	Headless UI primitives
class-variance-authority	0.7.1	Component variant management
clsx	2.1.1	Conditional class composition
tailwind-merge	3.4.0	Tailwind class conflict resolution

Global Styles and CSS Variables

The global and theme styles are in `src/theme/` only. The entry point is `src/theme/index.css`, with tokens split across `src/theme/tokens/`, base resets in `src/theme/base.css`, and component overrides in `src/theme/overrides/`.

```
</> 1 @import "tailwindcss";
2 @import "tw-animate-css";
3
4 @theme inline {
5   --radius-sm: calc(var(--radius) - 4px);
6   --radius-md: calc(var(--radius) - 2px);
7   --radius-lg: var(--radius);
8   --color-background: var(--background);
```

Contents

Technology Stack

Global Styles and CSS Variables

UI Component Architecture (shadcn/ui Pattern)

The cn() Utility

Component Styling Patterns

Responsive Design

Color Utility Enforcement

Reusing Styles: When to Extract



```
14
15 :root {
16   --radius: 0.625rem;
17   --background: #ffffff;
18   --foreground: #3f3f46;
19   --primary: #2563eb;
20   --primary-foreground: #ffffff;
21   --secondary: #f3f4f6;
22   --muted: #f9fafb;
23   --accent: #dbeafe;
24   --destructive: #b91c1c;
25   --border: #fafafa;
26   --ring: #3b82f6;
27   /* ... more tokens */
28 }
```

UI Component Architecture (shadcn/ui Pattern)

Components are stored in `src/components/ui/` and follow the shadcn/ui pattern.

- Built on Radix UI primitives for accessibility.
- Styled with Tailwind utility classes.
- Variants managed with `class-variance-authority` (cva).
- Classes composed using `cn()` utility (clsx + tailwind-merge).

The cn() Utility

A helper function that combines `clsx` for conditional classes with `tailwind-merge` to resolve conflicting utilities.

```
</>
1 // src/lib/utils.ts
2 import { clsx, type ClassValue } from 'clsx';
3 import { twMerge } from 'tailwind-merge';
4
5 export function cn(...inputs: ClassValue[]) {
6   return twMerge(clsx(inputs));
7 }
```

Component Styling Patterns

Style patterns for components include variant-based components with CVA, compound components, and direct utility class usage.

Variant-Based Components with CVA

With CVA, you can create variants of an existing component by extending its Tailwind classes without overriding them. Here's a minimal example.

[Try for free](#)

```
4 type ButtonVariant = "default" | "outline";
5 type ButtonSize = "default" | "sm";
6
7 const variantClasses: Record<ButtonVariant, string>
8   default: "bg-primary text-primary-foreground hover:
9   outline: "border bg-background hover:bg-accent hov
10 };
11
12 const sizeClasses: Record<ButtonSize, string> = {
13   default: "h-9 px-4 py-2",
14   sm: "h-8 px-3 text-xs",
15 };
16
17 function Button({
18   variant = "default",
19   size = "default",
20   className,
21   ...props
22 }: React.ComponentProps<"button"> & {
23   variant?: ButtonVariant;
24   size?: ButtonSize;
25 }) {
26   return (
27     <button
28       className={cn(
29         "inline-flex items-center justify-center rou
30         variantClasses[variant],
31         sizeClasses[size],
32         className,
33       )}
34       {...props}
35     />
36   );
37 }
```

Compound Components

Multi-part components are organized as separate functions:

```
</> | [icon]
1 // src/components/ui/card.tsx
2 function Card({ className, ...props }: React.Compone
3   return (
4     <div
5       className={cn(
6         "bg-card text-card-foreground flex flex-col
7         className,
8       )}
9       {...props}
10     />
11   );
12 }
13
14 function CardHeader({ className, ...props }: React.C
15   return (
16     <div
17       className={cn(
18         "@container/card-header grid auto-rows-min g
19         className,
20       )}
21       {...props}
22     />
```



Feature components apply Tailwind classes directly.

```
1 <Card
2   className={cn(
3     "group border rounded-xl overflow-hidden w-full
4     "flex flex-col-reverse h-full shadow-sm gap-0 py
5     "transition-all duration-200 hover:shadow-md",
6     className,
7   )}
8 >
9   {/* ... */}
10 </Card>
```

Responsive Design

Tailwind uses prefix-based responsive modifiers.

```
1 <div className="px-4 sm:px-6 lg:px-8 py-8">
2   <div className="grid md:grid-cols-2 gap-6">{/* Con
3     <div className="flex flex-col sm:flex-row gap-4 ju
4   </div>
```

Color Utility Enforcement

Hardcoded Tailwind color utilities like `bg-red` and `text-green` are blocked via an ESLint rule. Use semantic tokens (for example, `bg-primary`, `text-foreground`) or CSS variable classes instead.

Reusing Styles: When to Extract

Tailwind's utility-first approach means most styling lives inline in JSX. Before extracting a reusable abstraction, read the official guide on [managing reuse](#) – it covers multi-cursor editing, loops, and component extraction as the **preferred** strategies before reaching for CSS abstractions.

Use a React component (the default choice) when:

- The pattern involves markup structure—multiple elements, slots, children
- There is logic, state, or event handling
- It accepts props that change behavior or content
- It composes other components (shadcn, Radix, etc.)

Use a CSS component class (`@layer` components in `src/theme/base.css`) only when:



- It needs to be applied to many different HTML elements across the codebase (divs, sections, wrappers)
- Utilities need to override it in specific contexts (the components layer is lower specificity than utilities)

Example: `section-container` –consolidates `px-4 sm:px-8 lg:px-16 max-w-screen-2xl mx-auto` into one class, used by 30+ files. A page can add `max-w-4xl` alongside it and the utility wins.

Rule of thumb: if you can express it as a single `className` string with no JSX children, it's a CSS class. If it renders elements or accepts props, it's a React component.

```
</> 1 /* src/theme/base.css – CSS component class */
2 @layer components {
3   .section-container {
4     @apply px-4 sm:px-8 lg:px-16 max-w-screen-2xl mx
5   }
6 }
```

```
</> 1 /* React component – has structure, props, and child
2 function CategoryBanner({ title, image }: CategoryBa
3   return (
4     <div className="section-container">
5       <img src={image} alt="" />
6       <h1>{title}</h1>
7     </div>
8   );
9 }
```

Note

Don't use `@utility` for multi-property compositions that need to be overridable. The utility layer has the highest specificity, so any override attempt (e.g., adding `max-w-4xl` alongside a `@utility` class) would lose. Use `@layer` components instead.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

[Try for free](#)[Tableau](#)[Commerce Cloud](#)[Lightning Design System](#)[Einstein](#)[Quip](#)[APIs](#)[Trailhead](#)[Sample Apps](#)[AppExchange](#)[Partner Community](#)[Blog](#)[Salesforce Admins](#)[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#)[Terms of Service](#)[Privacy Information](#)[Responsible Disclosure](#)[Trust](#)[Contact](#)[Use of Cookies](#)[Cookie Preferences](#)[✔ X Your Privacy Choices](#)

